

AgentShield

Provider-independent runtime security and governance gateway for AI-agent tool actions

Team Member	Roll Number	Initial Idea
Avnish Raj	1024170125	SeatFlow
Laishram Amarjit	1024170129	AgentShield
Deepanjan Baral	1024170053	Profolio
Sameer Mathur	1024170132	Smart Interview Platform

Selected project: AgentShield Delivery target: 12-week academic MVP

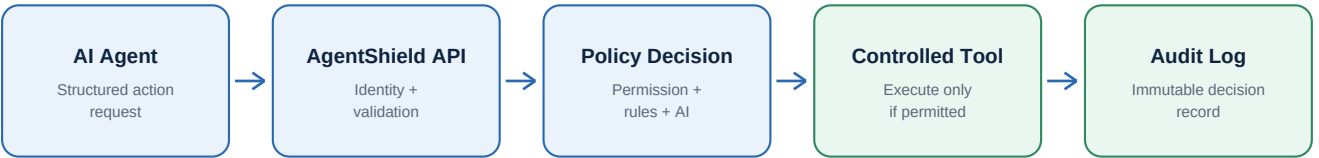
1. Problem and Motivation

Autonomous AI agents can request consequential actions such as sending emails, changing files, issuing refunds, or calling business APIs. Allowing an agent to execute these tools directly creates risks: excessive permissions, accidental or malicious actions, inconsistent policy enforcement, and weak accountability.

Organizations therefore need an independent control point that evaluates each action before execution and records the complete decision history.

2. Proposed Solution

AgentShield is a web-based governance gateway placed between an AI agent and controlled tools. The agent sends a structured action request to AgentShield instead of calling the tool directly. AgentShield authenticates the request, checks permissions, applies deterministic policies, optionally asks an AI model to interpret ambiguous risk, and returns one of three decisions: **ALLOW**, **BLOCK**, or **REQUIRE_APPROVAL**.

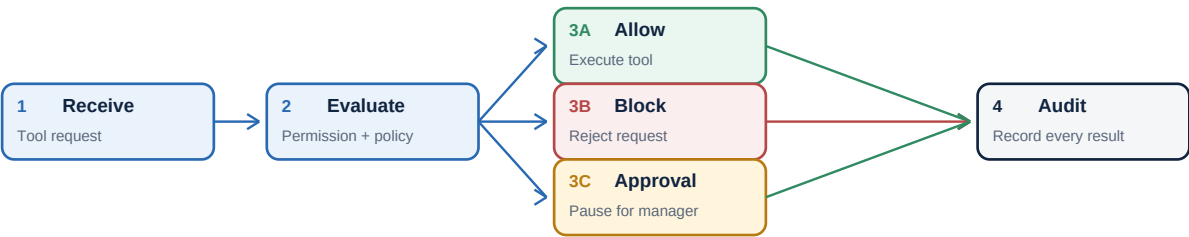


3. Intended Users and Objectives

USERS	PROJECT OBJECTIVES
User: submits or monitors agent actions. Manager: approves or rejects sensitive requests. Admin: manages agents, roles, policies and audit access.	- Prevent direct execution of unauthorized tool actions. - Apply consistent organizational rules. - Preserve human control over sensitive actions. - Maintain a searchable audit trail. - Demonstrate one provider integration without provider lock-in.

4. System Workflow

A request is evaluated once and branches to an outcome; ALLOW, BLOCK, and REQUIRE_APPROVAL are alternatives, not sequential steps. Every branch is audited. Approved requests return to the execution path, while rejected requests never reach the tool.



Action state model

RECEIVED -> EVALUATING -> ALLOWED / BLOCKED / PENDING_APPROVAL -> APPROVED / REJECTED -> EXECUTED / FAILED. An idempotency key prevents accidental duplicate execution.

5. MVP Features and Boundaries

Included in the 12-week MVP	Explicitly outside the MVP
Three roles: user, manager and admin	Automatic interception of existing Claude, Codex or Gemini applications
Agent registration and allowed tool scopes	OS, kernel or system-call interception
Central action-interception REST API	Real banking, payment or production refund systems
Deterministic permission and policy engine	Unrestricted access to real user files or company databases
Optional AI risk classification for ambiguous requests	Multiple AI providers in the first implementation
Human approval dashboard and status polling	MCP integration before the REST workflow is stable
Email, refund and sandbox-file simulators	Enterprise production-security certification
Append-only audit records and search	Public deployment before local testing is complete

6. Final Technology Stack

Layer	Selected Technology	Purpose
Frontend	React + Vite + JavaScript	Admin, manager and user dashboards
UI	Material UI	Forms, tables, dialogs and consistent components
Backend	Node.js + Express + JavaScript	Gateway API and workflow orchestration
Database	PostgreSQL + Prisma	Relational records, constraints, transactions and migrations
Validation	Zod	Validate every incoming action and policy payload
Security	JWT in HTTP-only cookies + bcrypt	Authentication, password hashing and role checks
AI	One OpenAI or Anthropic integration	Classify ambiguous risk; never replace critical rules
Updates	REST + 3-5 second polling	Simple and reliable approval-status refresh
Testing	Vitest, Supertest and Playwright	Unit, API and complete browser workflow tests
Local setup	Docker Compose	Repeatable application and PostgreSQL environment
Future	MCP and additional adapters	Provider/tool interoperability after the MVP

7. Demonstration Scenarios

Requested action	Policy	Expected result
Send a normal internal update	Internal non-sensitive email is permitted	ALLOW -> simulator executes -> audit
Issue a refund of Rs. 8,000	Refund above Rs. 5,000 needs manager approval	PENDING -> manager approves -> execute -> audit
Delete a protected file	Protected paths cannot be deleted	BLOCK -> no execution -> audit

8. AI Role and Defensibility

AgentShield is not merely an LLM wrapper. Its durable value comes from organizational permissions, deterministic policies, approval ownership, tool credentials and accumulated audit history. A generic AI model cannot independently know or enforce these local controls.

AI INTERPRETS	DETERMINISTIC CONTROLS ENFORCE
- Detect potentially confidential or ambiguous intent. - Assign Low, Medium or High risk. - Produce a short explanation and confidence value. - Assist when fixed rules are insufficient.	- Verify identity, role and tool scope. - Enforce refund limits and protected resources. - Require the correct human approval. - Prevent execution when validation or AI services fail. - Record every decision.

Fail-safe principle: if the AI API is unavailable, returns malformed output, or has low confidence, AgentShield does not automatically allow the action. It routes the request for human review or applies the stricter deterministic policy.

9. Validation and Quality Criteria

Area	Method	Acceptance criterion
Policy correctness	30-50 labelled allow/block/approval scenarios	All deterministic cases match expected decisions
Authorization	Role and tool-scope API tests	Unauthorized actions are rejected
Approval safety	State-transition and concurrency tests	No pending or rejected action can execute
Duplicate protection	Repeat the same idempotency key	At most one execution is created
Auditability	Compare requests, decisions and execution records	Every request has a complete audit entry
User workflow	Playwright tests for user, manager and admin	Core browser journeys pass consistently
AI resilience	Timeout, malformed response and offline tests	System fails closed and remains demonstrable

10. Twelve-Week Implementation Plan

Weeks	Focus	Deliverable
1-2	Foundation	Requirements, scenarios, database schema, authentication and roles
3-4	Gateway and policies	Action API, validation, permissions and deterministic rules
5	Controlled tools	Email, refund and sandbox-file simulators
6	Approval workflow	Pending queue, approve/reject and safe state transitions
7	Audit and dashboards	Searchable records and role-specific interfaces
8	AI layer	One provider adapter, structured output and fallback mode
9-10	Reliability	Unit, API, security and Playwright workflow testing
11	Evaluation and demo	Test results, seeded demo, documentation and presentation
12	Buffer	Bug fixing, final report and optional deployment

11. Feasibility, Risks and Conclusion

The academic MVP is feasible for four students within 12 weeks because it uses standard web technologies and controlled tool simulators. The main risks are scope expansion, incorrect approval-state handling, dependence on a live AI API, and insecure credential management. These are reduced through a fixed MVP boundary, PostgreSQL transactions, idempotency, automated tests, environment-based secrets, restricted cookie settings, and an offline demonstration fallback.

Conclusion: AgentShield will demonstrate a complete and measurable security workflow: intercept an AI-agent action, evaluate permissions and policy, request human approval when required, execute only permitted simulated tools, and preserve an audit trail. Deployment, MCP and additional provider adapters will be considered only after the local MVP is stable.